**EX NO : 1 ANALYSIS OF NYC TAXI TRIP DATA USING APACHE SPARK AND PYTHON VISUALIZATION TOOLS**

**DATE :**

**AIM**

To analyse NYC Taxi trip data using Apache Spark and Python visualization tools to identify usage patterns, trip characteristics, and peak travel times.

**ALGORITHM / PROCEDURE:**

1. Start the Apache Spark environment by creating a SparkSession.
2. Load the NYC Taxi CSV dataset into a Spark DataFrame using spark.read.csv().
3. Display the dataset schema and sample records to understand the data structure.
4. Perform data cleaning by removing records with invalid trip distance and fare values.
5. Perform feature engineering by:
    - Calculating trip duration using pickup and drop-off timestamps.
    - Extracting pickup hour and pickup day from datetime fields.
6. Perform analytical operations using Spark:
    - Calculate total trip count per hour.
    - Compute average fare per hour.
    - Compute average trip distance per day.
7. Convert Spark DataFrames into Pandas DataFrames for visualization.
8. Save the processed analytical results as CSV files.
9. Stop the Spark session.

**#Installation commands in terminal**

```
pip3 install pyspark
pyspark
sudo apt update
sudo apt install libblas-dev liblapack-dev

#In terminal, install these for visualization
pip install pyspark pandas matplotlib seaborn
```

**Code:**

**nyc_taxi_analysis.py**

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, hour, dayofweek, unix_timestamp

# 1. Create Spark Session
```

```
spark = SparkSession.builder \
    .appName("NYC Taxi Pattern Analysis") \
    .getOrCreate()


# 2. Load Dataset

data_path = "/home/sys50/Documents/data.csv"
df = spark.read.csv(data_path, header=True, inferSchema=True)
print("Schema:")
df.printSchema()
df.show(5)


# 3. Data Cleaning
df = df.filter(
    (col("trip_distance") > 0) &
    (col("fare_amount") > 0)
)

# 4. Feature Engineering

df = df.withColumn(
    "trip_duration_minutes",
    (unix_timestamp("tpep_dropoff_datetime") -
    unix_timestamp("tpep_pickup_datetime")) / 60
)

df = df.withColumn("pickup_hour", hour("tpep_pickup_datetime"))
df = df.withColumn("pickup_day", dayofweek("tpep_pickup_datetime"))


# 5. Spark Analysis

trip_count_hour = df.groupBy("pickup_hour").count().orderBy("pickup_hour")
avg_fare_hour = df.groupBy("pickup_hour").avg("fare_amount").orderBy("pickup_hour")
avg_distance_day = df.groupBy("pickup_day").avg("trip_distance").orderBy("pickup_day")


# 6. Convert to Pandas

trip_count_hour_pd = trip_count_hour.toPandas()
avg_fare_hour_pd = avg_fare_hour.toPandas()
avg_distance_day_pd = avg_distance_day.toPandas()

# Small sample for scatter plot
df_pd = df.select("trip_distance", "fare_amount").sample(0.01).toPandas()

# 7. Save Outputs for Visualization

output_path = "/home/sys50/Documents/"
trip_count_hour_pd.to_csv(output_path + "trip_count_hour.csv", index=False)
```

```
avg_fare_hour_pd.to_csv(output_path + "avg_fare_hour.csv", index=False)
avg_distance_day_pd.to_csv(output_path + "avg_distance_day.csv", index=False)
df_pd.to_csv(output_path + "distance_fare_sample.csv", index=False)

print("Analysis CSV files saved successfully.")
spark.stop()
```

**OUTPUT:**

```
+------+--------------+--------------+----------+---------+-------+------------+--
--------+---------+---------+-------+---+---------+------------+---------------+--------------+-------
-----+-------------+
|VendorID|tpep_pickup_datetime|tpep_dropoff_datetime|passenger_count|trip_distance|RatecodeID|
store_and_fwd_flag|PULocationID|DOLocationID|payment_type|fare_amount|extra|mta_tax|tip_am
ount|tolls_amount|improvement_surcharge|total_amount|congestion_surcharge|
+------+--------------+--------------+----------+---------+-------+------------+--
--------+---------+---------+-------+---+---------+------------+---------+--------------+-------
---- +-------------+
|     1| 2020-01-01 00:28:15|  2020-01-01 00:33:03|              1|          1.2|         1|                N|
238|       239|         1|       6.0| 3.0|   0.5|      1.47|       0.0|              0.3|     11.27|              2.5|
|     1| 2020-01-01 00:35:39|  2020-01-01 00:43:04|              1|          1.2|         1|                N|
239|       238|         1|       7.0| 3.0|   0.5|       1.5|       0.0|              0.3|     12.3|              2.5|
|     1| 2020-01-01 00:47:41|  2020-01-01 00:53:52|              1|          0.6|         1|                N|
238|       238|         1|       6.0| 3.0|   0.5|       1.0|       0.0|              0.3|     10.8|              2.5|
|     1| 2020-01-01 00:55:23|  2020-01-01 01:00:14|              1|          0.8|         1|                N|
238|       151|         1|       5.5| 0.5|   0.5|      1.36|       0.0|              0.3|     8.16|              0.0|
|     2| 2020-01-01 00:01:58|  2020-01-01 00:04:16|              1|          0.0|         1|                N|
193|       193|         2|       3.5| 0.5|   0.5|       0.0|       0.0|              0.3|      4.8|              0.0|
+------+--------------+--------------+----------+---------+-------+------------+--
--------+---------+---------+-------+---+---------+------------+---------+--------------+-------
-----+-------------+
 only showing top 5 rows

Analysis CSV files saved successfully.
```

**ALGORITHM / PROCEDURE FOR VISUALIZATION:**

1. Import Pandas, Matplotlib, and Seaborn libraries.
2. Load the CSV files generated from Spark analysis.
3. Plot a line chart to show taxi demand by pickup hour.
4. Plot a bar chart to visualize average fare by hour.
5. Plot a bar chart to visualize average trip distance by day.
6. Plot a scatter plot to show the relationship between trip distance and fare amount.
7. Plot a box plot to analyze fare distribution and identify outliers.
8. Display all visualizations.

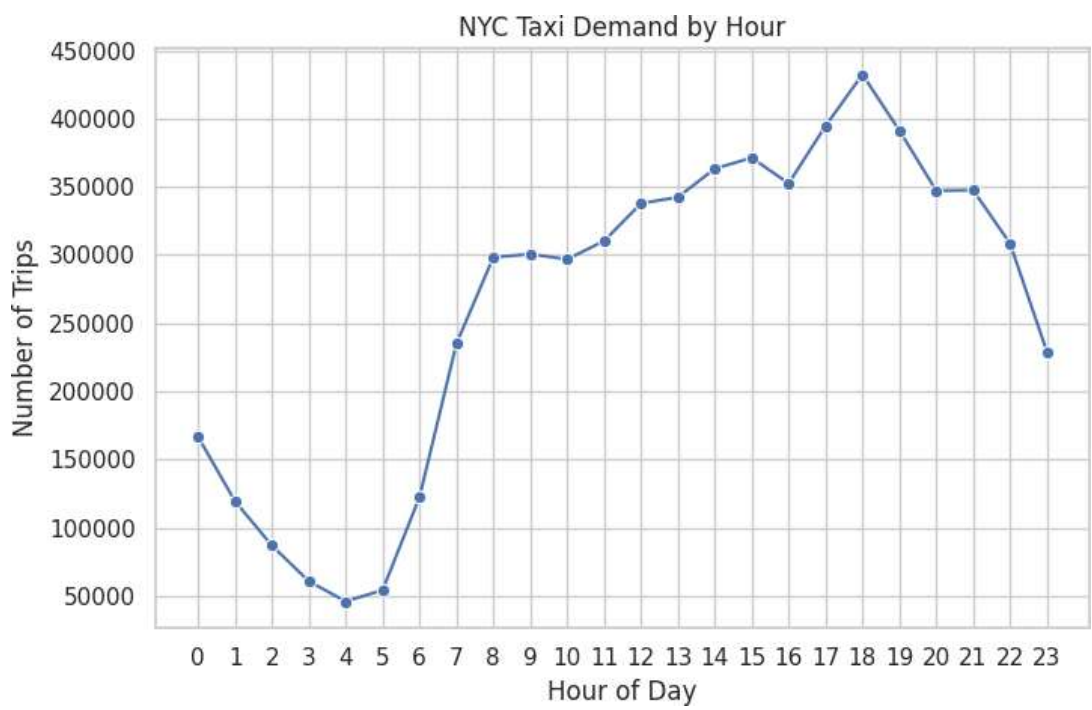
**Code For visualization**

**nyc_taxi_visual.py**

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.set(style="whitegrid")

# Load CSV files generated by Spark
base_path = "/home/sys50/Documents/"

trip_count_hour_pd = pd.read_csv(base_path + "trip_count_hour.csv")
avg_fare_hour_pd = pd.read_csv(base_path + "avg_fare_hour.csv")
avg_distance_day_pd = pd.read_csv(base_path + "avg_distance_day.csv")
df_pd = pd.read_csv(base_path + "distance_fare_sample.csv")

# 1. Peak Taxi Usage Time
plt.figure(figsize=(8,5))
sns.lineplot(
    x=trip_count_hour_pd["pickup_hour"],
    y=trip_count_hour_pd["count"],
    marker="o"
)
plt.xlabel("Hour of Day")
plt.ylabel("Number of Trips")
plt.title("NYC Taxi Demand by Hour")
plt.xticks(range(0,24))
plt.show()

# 2. Average Fare by Hour
plt.figure(figsize=(8,5))
sns.barplot(
    x=avg_fare_hour_pd["pickup_hour"],
    y=avg_fare_hour_pd["avg(fare_amount)"]
)
plt.xlabel("Hour of Day")
plt.ylabel("Average Fare Amount ($)")
plt.title("Average Fare by Pickup Hour")
plt.xticks(range(0,24))
plt.show()

# 3. Average Trip Distance by Day
plt.figure(figsize=(8,5))
sns.barplot(
    x=avg_distance_day_pd["pickup_day"],
    y=avg_distance_day_pd["avg(trip_distance)"]
)
plt.xlabel("Day of Week (1 = Sunday)")
plt.ylabel("Average Trip Distance (miles)")
plt.title("Average Trip Distance by Day")
plt.show()
```

```
# 4. Trip Distance vs Fare
plt.figure(figsize=(7,5))
sns.scatterplot(
    x=df_pd["trip_distance"],
    y=df_pd["fare_amount"],
    alpha=0.5
)
plt.xlabel("Trip Distance (miles)")
plt.ylabel("Fare Amount ($)")
plt.title("Trip Distance vs Fare Amount")
plt.show()

# 5. Fare Distribution
plt.figure(figsize=(7,5))
sns.boxplot(y=df_pd["fare_amount"])
plt.ylabel("Fare Amount ($)")
plt.title("Fare Amount Distribution")
plt.show()
```

**OUTPUT:**




**RESULT:**

Thus the analysis of NYC Taxi trip data using Apache Spark and Python visualization tools to identify usage patterns, trip characteristics, and peak travel times was successfully executed.

**Ex No: 2 DEVELOP A PREDICTIVE MODEL FOR SOCIAL MEDIA ENGAGEMENT**
**DATE:**

**AIM**

To develop a predictive model for social media engagement using Apache Spark, ML lib and PySpark

**ALGORITHM / PROCEDURE:**

1. Initialize a Spark session using SparkSession.
2. Load the social media dataset into a Spark DataFrame.
3. Display schema and sample records.
4. Encode categorical string columns (post_id, day_of_week, location) using StringIndexer.
5. Select feature columns and define the target label.
6. Combine feature columns into a single feature vector using VectorAssembler.
7. Normalize the feature vector using StandardScaler.
8. Split the dataset into training and testing sets.
9. Train a Linear Regression model using the training dataset.
10. Predict engagement values using the test dataset.
11. Evaluate the model performance using RMSE.
12. Display prediction results and RMSE value.
13. Stop the Spark session.

**Code:**

```
from pyspark.sql import SparkSession
from pyspark.ml.feature import VectorAssembler, StandardScaler, StringIndexer
from pyspark.ml.regression import LinearRegression
from pyspark.ml.evaluation import RegressionEvaluator

# Step 1: Spark Session
spark = SparkSession.builder \
    .appName("Social Media Engagement Prediction") \
    .getOrCreate()

# Step 2: Load Dataset
data_path = "/home/sys50/Documents/dataset.csv"
df = spark.read.csv(data_path, header=True, inferSchema=True)

df.printSchema()
df.show(5)
```

```
# Step 3: Encode STRING columns
post_indexer = StringIndexer(inputCol="post_id", outputCol="post_id_index")
day_indexer = StringIndexer(inputCol="day_of_week", outputCol="day_index")
loc_indexer = StringIndexer(inputCol="location", outputCol="location_index")

df = post_indexer.fit(df).transform(df)
df = day_indexer.fit(df).transform(df)
df = loc_indexer.fit(df).transform(df)

# Step 4: Define Features & Label
features = ["post_id_index", "day_index", "location_index"]
label = "engagement_rate"

# Step 5: Vector Assembler
assembler = VectorAssembler(
    inputCols=features,
    outputCol="features_vector"
)

df = assembler.transform(df)

# Step 6: Feature Scaling
scaler = StandardScaler(
    inputCol="features_vector",
    outputCol="scaled_features",
    withStd=True,
    withMean=False
)

scalerModel = scaler.fit(df)
df = scalerModel.transform(df)

# Step 7: Train-Test Split
train_data, test_data = df.randomSplit([0.8, 0.2])

# Step 8: Linear Regression Model
lr = LinearRegression(
    featuresCol="scaled_features",
    labelCol=label
)

lrModel = lr.fit(train_data)

# Step 9: Prediction
predictions = lrModel.transform(test_data)
predictions.select("prediction",  label).show(5)

# Step 10: Evaluation
evaluator = RegressionEvaluator(
    labelCol=label,
    predictionCol="prediction",
    metricName="rmse"
```

```
)

rmse = evaluator.evaluate(predictions)
print(f"Root Mean Squared Error (RMSE) on test data = {rmse}")

spark.stop()
```

**OUTPUT:**

```
+-----------+----------+
|        prediction|engagement_rate|
+-----------+----------+
| 0.2652637112703293|         0.02507|
|0.24967958929629827|         0.04266|
| 0.2337108508206715|         0.03216|
|0.23270588080314936|         0.14015|
|0.26447970714822344|         0.4481|
+--------------------+----------+
only showing top 5 rows

Root Mean Squared Error (RMSE) on test data = 1.5248060459433126
```

**RESULT:**

Thus, the above program Develop a Predictive Model for Social Media Engagement was successfully executed.

**EX.NO:3 INTEGRATE STATISTICAL MEASURES AND PROBABILITY ANALYSIS**
**DATE:**

**AIM:**

To perform statistical analysis and apply probability theories to a dataset, covering mean, median, mode, standard deviation, variance, PDF, percentiles, moments, correlation, covariance, and Bayes' Theorem application.

**ALGORITHM / PROCEDURE:**

1. Import required Python libraries such as Pandas, NumPy, Matplotlib, Seaborn, and SciPy.
2. Load the dataset using Pandas.
3. Convert pickup and drop-off timestamps into datetime format.
4. Calculate trip duration in minutes.
5. Remove invalid records with negative or zero values.
6. Select numerical attributes for analysis.
7. Compute descriptive statistics:
    - Mean
    - Median
    - Mode
    - Variance
    - Standard deviation
8. Calculate percentiles for fare amount.
9. Compute statistical moments such as skewness and kurtosis.
10. Compute correlation and covariance matrices.
11. Visualize data using:
    - Histogram with KDE
    - Box plot
    - Correlation heatmap
12. Compute and plot the Probability Density Function (PDF).
13. Apply Bayes' Theorem to calculate conditional probability.
14. Display all computed results and plots.

**CODE**

**#installation command**

```
#type the below installation command in terminal
#pip install pandas numpy matplotlib seaborn scipy
```

**statistical_analysis.py**

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

sns.set_style(style="whitegrid")

# 1. Load Dataset

data_path = "/home/sys50/Documents/data.csv"
df = pd.read_csv(data_path, low_memory=False)

print(df.head())
print(df.describe())

# 2. Convert datetime & create trip_duration

df["tpep_pickup_datetime"] = pd.to_datetime(df["tpep_pickup_datetime"])
df["tpep_dropoff_datetime"] = pd.to_datetime(df["tpep_dropoff_datetime"])

df["trip_duration"] = (
   df["tpep_dropoff_datetime"] - df["tpep_pickup_datetime"]
).dt.total_seconds() / 60  # minutes

# Remove invalid values
df = df[
   (df["trip_distance"] > 0) &
   (df["fare_amount"] > 0) &
   (df["trip_duration"] > 0)
]

# 3. Select Numeric Columns

data = df[["trip_distance", "fare_amount", "trip_duration"]]

# 4. Descriptive Statistics
print("\nMean:\n", data.mean())
print("\nMedian:\n", data.median())
print("\nMode:\n", data.mode().iloc[0])
print("\nVariance:\n", data.var())
print("\nStandard Deviation:\n", data.std())
```

```
# 5. Percentiles

print("\nFare Amount Percentiles:",
    np.percentile(data["fare_amount"], [25, 50, 75, 90]))

# 6. Moments

print("\nSkewness:\n", data.skew())
print("\nKurtosis:\n", data.kurtosis())

# 7. Correlation & Covariance

print("\nCorrelation Matrix:\n", data.corr())
print("\nCovariance Matrix:\n", data.cov())

# 8. Visualizations
plt.figure(figsize=(7,5))
sns.histplot(data["fare_amount"], kde=True)
plt.title("Fare Amount Distribution")
plt.show()

plt.figure(figsize=(7,5))
sns.boxplot(y=data["fare_amount"])
plt.title("Fare Amount Boxplot")
plt.show()

plt.figure(figsize=(7,5))
sns.heatmap(data.corr(), annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()


# 9. Probability Density Function
x = np.linspace(
   data["fare_amount"].min(),
   data["fare_amount"].max(), 100
)
pdf = stats.norm.pdf(
   x,
   data["fare_amount"].mean(),
   data["fare_amount"].std()
)

plt.figure(figsize=(7,5))
plt.plot(x, pdf)
plt.title("PDF of Fare Amount")
plt.xlabel("Fare Amount")
plt.ylabel("Density")
plt.show()

# 10. Bayes Theorem Result:")
print("P(High Fare | Long Distance) =", P_high_given_long)
```

**OUTPUT:**

```
 VendorID tpep_pickup_datetime tpep_dropoff_datetime  passenger_count  trip_distance  ...
tip_amount tolls_amount  improvement_surcharge  total_amount  congestion_surcharge
0       1.0  2020-01-01 00:28:15   2020-01-01 00:33:03            1.0           1.2  ...       1.47          0.0
0.3         11.27                 2.5
1       1.0  2020-01-01 00:35:39   2020-01-01 00:43:04            1.0           1.2  ...       1.50          0.0
0.3         12.30                 2.5
2       1.0  2020-01-01 00:47:41   2020-01-01 00:53:52            1.0           0.6  ...       1.00          0.0
0.3         10.80                 2.5
3       1.0  2020-01-01 00:55:23   2020-01-01 01:00:14            1.0           0.8  ...       1.36          0.0
0.3          8.16                 0.0
4       2.0  2020-01-01 00:01:58   2020-01-01 00:04:16            1.0           0.0  ...       0.00          0.0
0.3          4.80                 0.0

[5 rows x 18 columns]
          VendorID  passenger_count  trip_distance    RatecodeID  PULocationID  ...    tip_amount
tolls_amount  improvement_surcharge  total_amount  congestion_surcharge
count  6.339567e+06     6.339567e+06  6.405008e+06  6.339567e+06  6.405008e+06  ...
6.405008e+06  6.405008e+06           6.405008e+06  6.405008e+06          6.405008e+06
mean   1.669624e+00     1.515333e+00  2.929644e+00  1.059908e+00  1.647323e+02  ...
2.189342e+00  3.488395e-01           2.979870e-01  1.866315e+01          2.275662e+00
std    4.703484e-01     1.151594e+00  8.315911e+01  8.118432e-01  6.554374e+01  ...
2.760028e+00  1.766978e+00           3.385937e-02  1.475736e+01          7.352646e-01
min    1.000000e+00     0.000000e+00  -3.062000e+01  1.000000e+00  1.000000e+00  ... -
9.100000e+01 -3.574000e+01          -3.000000e-01 -1.242300e+03         -2.500000e+00
25%    1.000000e+00     1.000000e+00   9.600000e-01  1.000000e+00  1.320000e+02  ...
0.000000e+00  0.000000e+00           3.000000e-01  1.116000e+01          2.500000e+00
50%    2.000000e+00     1.000000e+00   1.600000e+00  1.000000e+00  1.620000e+02  ...
1.950000e+00  0.000000e+00           3.000000e-01  1.430000e+01          2.500000e+00
75%    2.000000e+00     2.000000e+00   2.930000e+00  1.000000e+00  2.340000e+02  ...
2.860000e+00  0.000000e+00           3.000000e-01  1.980000e+01          2.500000e+00
max    2.000000e+00     9.000000e+00  2.102401e+05  9.900000e+01  2.650000e+02  ...
1.100000e+03  9.105000e+02           3.000000e-01  4.268300e+03          2.750000e+00

[8 rows x 15 columns]
```

**Mean:**

```
 trip_distance     2.967762
fare_amount       12.597843
trip_duration     16.011132
dtype: float64
```

**Median:**

```
 trip_distance     1.610000
fare_amount        9.000000
trip_duration     10.533333
dtype: float64
```

**Mode:**

```
 trip_distance     0.9
```

```
fare_amount      6.0
trip_duration    7.0
Name: 0, dtype: float64
```

**Variance:**

```
 trip_distance    7012.752368
fare_amount       131.591300
trip_duration    4004.834803
dtype: float64
```

**Standard Deviation:**

```
 trip_distance    83.742178
fare_amount       11.471325
trip_duration    63.283764
dtype: float64
```

**Fare Amount Percentiles:** [ 6.5  9.  14.  24. ]

**Skewness:**

```
 trip_distance    2505.292814
fare_amount        12.641428
trip_duration      21.836233
dtype: float64
```

**Kurtosis:**

```
 trip_distance    6.289608e+06
fare_amount       3.132618e+03
trip_duration    5.353239e+02
dtype: float64
```

**Correlation Matrix:**

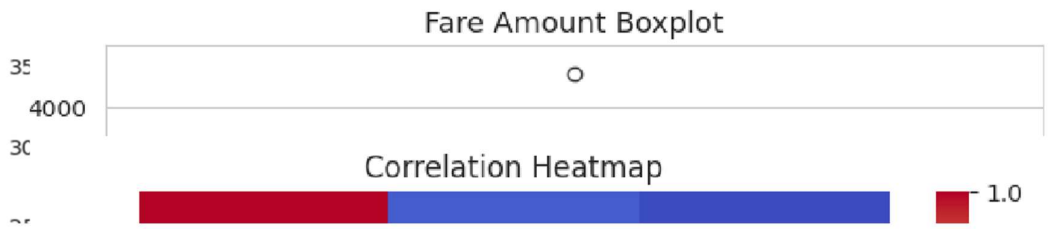
| | trip_distance | fare_amount | trip_duration |
|---|---|---|---|
| trip_distance | 1.000000 | 0.042821 | 0.006628 |
| fare_amount | 0.042821 | 1.000000 | 0.151033 |
| trip_duration | 0.006628 | 0.151033 | 1.000000 |

**Covariance Matrix:**

| | trip_distance | fare_amount | trip_duration |
|---|---|---|---|
| trip_distance | 7012.752368 | 41.135552 | 35.123890 |
| fare_amount | 41.135552 | 131.591300 | 109.642409 |
| trip_duration | 35.123890 | 109.642409 | 4004.834803 |

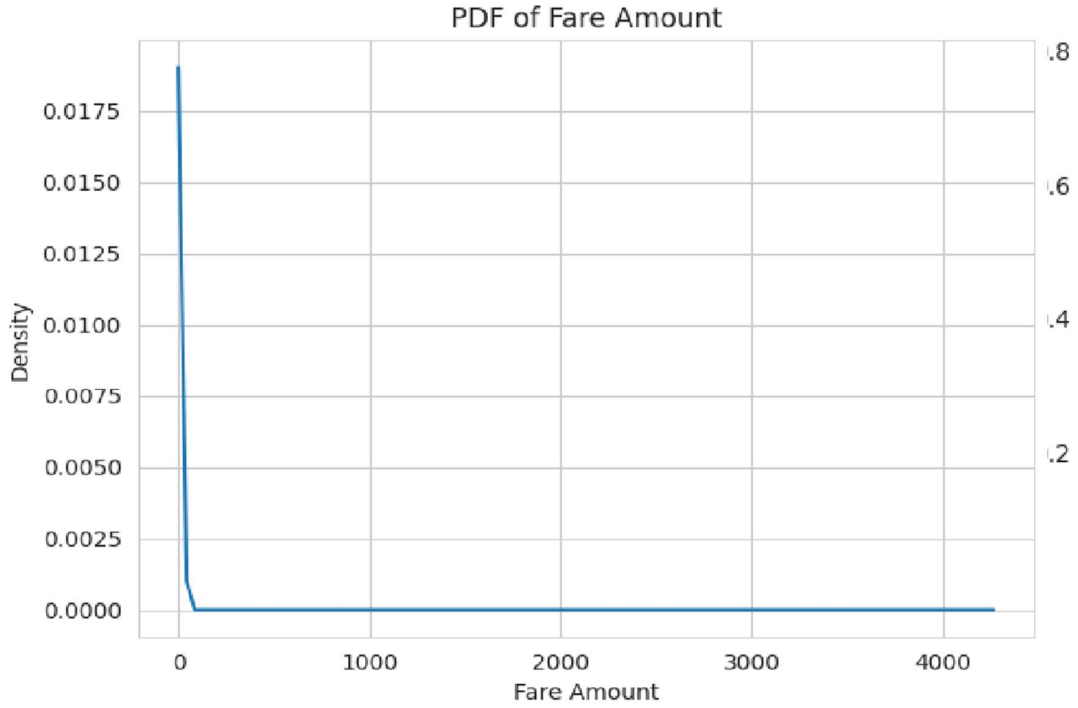
**Bayes' Theorem Result:**

P(High Fare | Long Distance) = 0.8764374261755049










**RESULT:**

Thus, the above program Integrate Statistical Measures and Probability Analysis was successfully executed.

**EX.NO:4 IMPLEMENT AND COMPARE DIMENSIONAL REDUCTION METHODS**
**DATE :**

**AIM**

To Implement and Compare Dimensionality Reduction Methods: focus on the necessity of dimensionality reduction, executing PCA and LDA to reduce dataset dimensions, enhance class separability, and conduct a comparitive analysis and multivariate analysis post-reduction

**ALGORITHM/PROCEDURE:**

1. Import required libraries including NumPy, Pandas, Matplotlib, and Scikit-learn.
2. Load the Iris dataset.
3. Separate features and class labels.
4. Standardize the feature values using StandardScaler.
5. Apply Principal Component Analysis (PCA) to reduce dimensions.
6. Transform the dataset using PCA components.
7. Apply Linear Discriminant Analysis (LDA) using class labels.
8. Transform the dataset using LDA components.
9. Plot PCA and LDA projections for visual comparison.
10. Split the reduced datasets into training and testing sets.
11. Train Logistic Regression models on PCA and LDA reduced data.
12. Evaluate classification accuracy for both PCA and LDA.
13. Compare the performance results.

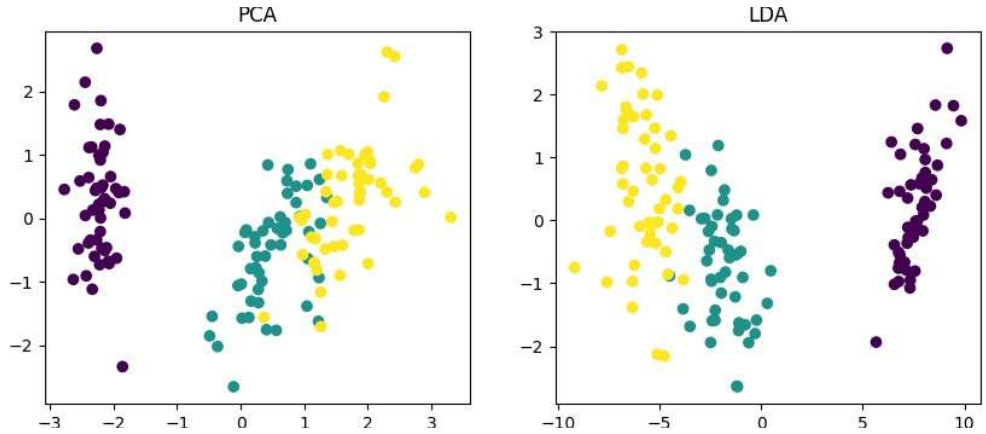
**CODE**
**PCA_LDA.py**

```
# installation command in terminal
# pip install numpy pandas matplotlib seaborn scikit-learn

import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
# Load the Iris dataset
iris = load_iris()
X = iris.data
y = iris.target
#2. Preprocess the Data
from sklearn.preprocessing import StandardScaler
# Standardize the features
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
#3. Apply PCA and LDA for Dimensionality Reduction
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
# Apply PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
# Apply LDA
lda = LinearDiscriminantAnalysis(n_components=2)
X_lda = lda.fit_transform(X_scaled, y)
#4. Analyze the Results and Compare the Performance
import matplotlib.pyplot as plt
# Plot PCA
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap='viridis')
plt.title('PCA')
# Plot LDA
plt.subplot(1, 2, 2)
plt.scatter(X_lda[:, 0], X_lda[:, 1], c=y, cmap='viridis')
plt.title('LDA')
plt.show()
#5. Perform Multivariate Analysis Post-Reduction
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.linear_model import LogisticRegression
# Split data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X_pca, y, test_size=0.2, random_state=42)
# Train logistic regression on PCA-reduced data
model_pca = LogisticRegression()
model_pca.fit(X_train, y_train)
y_pred_pca = model_pca.predict(X_test)
accuracy_pca = accuracy_score(y_test, y_pred_pca)
print("Accuracy on PCA-reduced data:", accuracy_pca)
# Split data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X_lda, y, test_size=0.2, random_state=42)
# Train logistic regression on LDA-reduced data
model_lda = LogisticRegression()
model_lda.fit(X_train, y_train)
y_pred_lda = model_lda.predict(X_test)
accuracy_lda = accuracy_score(y_test, y_pred_lda)
print("Accuracy on LDA-reduced data:", accuracy_lda)
```

**OUTPUT**

```
Accuracy on PCA-reduced data: 0.9
Accuracy on LDA-reduced data: 1.0
```




**RESULT**:

Thus, the above program to Implement and Compare Dimensional Reduction Methods was successfully executed.

**EX.NO:5 IMPLEMENT REGRESSION TECHNIQUES AND DATA PER-PROCESSING**
**DATE:**

AIM

To incorporate data preprocessing techniques and to apply linear, polynomial, and multivariate regression models for accurate prediction.

**ALGORITHM/PROCEDURE:**

1. Create a dataset with multiple numerical attributes.
2. Handle missing values by replacing them with median values.
3. Remove duplicate records from the dataset.
4. Normalize numerical features using standardization.
5. Detect outliers using Isolation Forest algorithm.
6. Display the outlier detection table.
7. Split the dataset into training and testing sets.
8. Train a Linear Regression model and evaluate its RMSE.
9. Apply Polynomial Regression for nonlinear modeling.
10. Evaluate Polynomial Regression using RMSE.
11. Compare the performance of Linear and Polynomial Regression models.

**CODE:**

```
import numpy as np
import pandas as pd

from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from sklearn.ensemble import IsolationForest

# Step 1: Dataset
data = {
   'SquareFeet': [650, 850, np.nan, 1100, 950, 1170, 980, np.nan, 700, 850, 1000, 960, 850],
   'Bedrooms': [2, 3, 2, 3, np.nan, 4, 3, 4, 2, 3, 3, 2, 3],
   'Age': [5, 7, 3, 10, 2, 12, 8, 4, 5, 6, 9, 11, 7],
   'Price': [300000, 350000, 320000, 500000, 330000, 600000,
            370000, 620000, 310000, 340000, 400000, 360000, 350000]
}

df = pd.DataFrame(data)

# Step 2: Missing Values
for col in ['SquareFeet', 'Bedrooms', 'Age']:
```

```
    df[col] = df[col].fillna(df[col].median())

# Step 3: Remove Duplicates
df.drop_duplicates(inplace=True)

# Step 4: Standardization
scaler = StandardScaler()
df[['SquareFeet', 'Bedrooms', 'Age']] = scaler.fit_transform(
    df[['SquareFeet', 'Bedrooms', 'Age']]
)

# Step 5: Isolation Forest
iso = IsolationForest(n_estimators=100, contamination=0.15, random_state=42)
df['outliers'] = iso.fit_predict(df[['SquareFeet', 'Bedrooms', 'Age']])

# Output 1: Full Outlier Table
print("\nFULL OUTLIER DETECTION TABLE:\n")
print(df.reset_index())

# Output 2: Only Detected Outliers
outliers_only = df[df['outliers'] == -1][
    ['SquareFeet', 'Bedrooms', 'Age', 'Price', 'outliers']
]

print("\nOutliers detected using Isolation Forest:")
print(outliers_only)

# - - - - - - - - - - - - - - - - - - -
# Step 6: Linear Regression
# - - - - - - - - - - - - - - - - - - -
X = df[['SquareFeet', 'Bedrooms', 'Age']]
y = df['Price']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

lr = LinearRegression()
lr.fit(X_train, y_train)

y_pred = lr.predict(X_test)

print("\nLinear Regression RMSE:",
      np.sqrt(mean_squared_error(y_test, y_pred)))

# Step 7: Polynomial Regression
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)

X_train_p, X_test_p, y_train_p, y_test_p = train_test_split(
    X_poly, y, test_size=0.2, random_state=42
)
```

```
poly_reg = LinearRegression()
poly_reg.fit(X_train_p, y_train_p)

y_pred_p = poly_reg.predict(X_test_p)

print("Polynomial Regression RMSE:",
    np.sqrt(mean_squared_error(y_test_p, y_pred_p)))
```

**OUTPUT:**

FULL OUTLIER DETECTION TABLE:

| | index | SquareFeet | Bedrooms | Age | Price | outliers |
|---|---|---|---|---|---|---|
| 0 | 0 | -1.950877 | -1.212678 | -0.595683 | 300000 | -1 |
| 1 | 1 | -0.536344 | 0.242536 | 0.054153 | 350000 | 1 |
| 2 | 2 | 0.170923 | -1.212678 | -1.245520 | 320000 | 1 |
| 3 | 3 | 1.231822 | 0.242536 | 1.028908 | 500000 | 1 |
| 4 | 4 | 0.170923 | 0.242536 | -1.570438 | 330000 | 1 |
| 5 | 5 | 1.726909 | 1.697749 | 1.678744 | 600000 | -1 |
| 6 | 6 | 0.383103 | 0.242536 | 0.379071 | 370000 | 1 |
| 7 | 7 | 0.170923 | 1.697749 | -0.920602 | 620000 | 1 |
| 8 | 8 | -1.597243 | -1.212678 | -0.595683 | 310000 | 1 |
| 9 | 9 | -0.536344 | 0.242536 | -0.270765 | 340000 | 1 |
| 10 | 10 | 0.524556 | 0.242536 | 0.703989 | 400000 | 1 |
| 11 | 11 | 0.241649 | -1.212678 | 1.353826 | 360000 | 1 |

Outliers detected using Isolation Forest:

| | SquareFeet | Bedrooms | Age | Price | outliers |
|---|---|---|---|---|---|
| 0 | -1.950877 | -1.212678 | -0.595683 | 300000 | -1 |
| 5 | 1.726909 | 1.697749 | 1.678744 | 600000 | -1 |

Linear Regression RMSE: 53952.69437438349 Polynomial Regression RMSE: 37736.92624527236

**RESULT:**

Thus, the above program for Implementing Regression Techniques and Data Pre-processing was successfully executed.

**EX.NO:6** **Explore machine learning techniques with real-world data**

**DATE:**

**AIM**

To understand and apply machine learning paradigms (supervised, unsupervised, reinforcement learning) for real-world data challenges, including K-fold cross-validation and clustering techniques.

**ALGORITHM / PROCEDURE**

1. **Importing Libraries**: Import necessary libraries including NumPy, pandas, Matplotlib, Seaborn, and scikit-learn modules.
2. **Loading Dataset**: Load the Iris dataset using load_iris() function from scikit-learn. Split the dataset into features (X) and target labels (y).
3. **Data Splitting**: Split the data into training and testing sets using train_test_split () function from scikit-learn. Here, 80% of the data is used for training and 20% for testing.
4. **Supervised Learning - Linear Regression**: Train a linear regression model (linear_reg_model) using the training data (X_train, y_train).
5. **Supervised Learning - Logistic Regression**: Train a logistic regression model (logreg_model) using the training data (X_train, y_train).
6. **K-fold Cross-Validation**: Perform K-fold cross-validation with a decision tree classifier (tree_model) using cross_val_score () function. Here, K is set to 5.
7. **Unsupervised Learning - K-Means Clustering**: Standardize the features using StandardScaler () from scikit-learn. Fit a K-means clustering model (kmeans_model) with 3 clusters to the standardized data.
8. **Evaluation: Silhouette Score**: Calculate the silhouette score for K-means clustering using silhouette_score() from scikit-learn.
9. **Plotting Results**: Plot the results using Matplotlib. Two subplots are created side by side:
   - The left subplot shows the actual versus predicted values for the linear regression model.
   - The right subplot shows the actual versus predicted values for the logistic regression model.

**Display Plots**: Display the plots using plt.show()

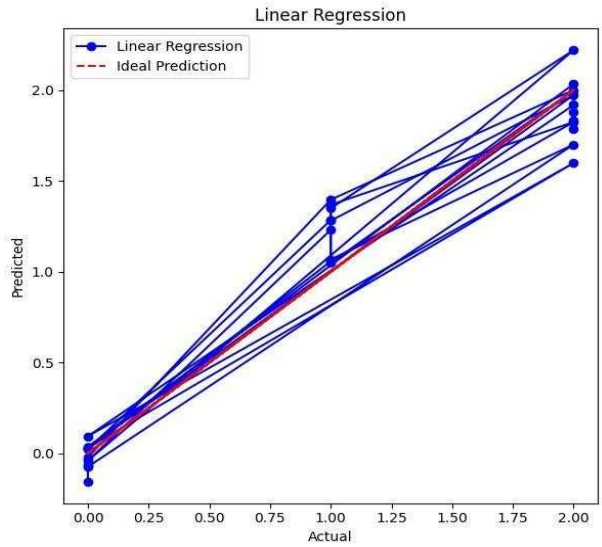
**PROGRAM:**

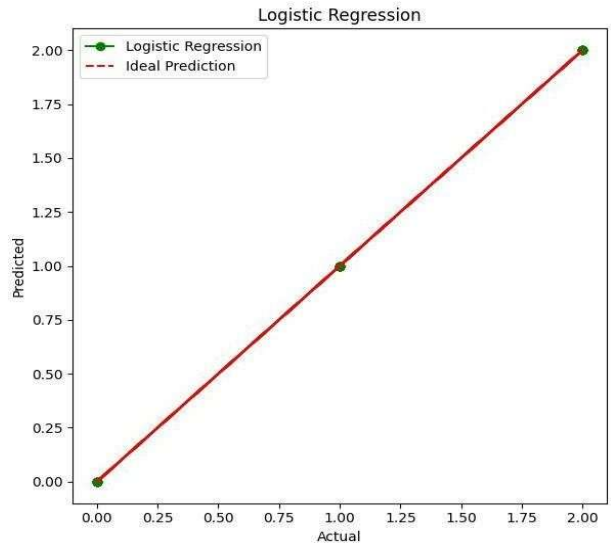
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.cluster import KMeans
from sklearn.metrics import  silhouette_score
from sklearn.preprocessing import StandardScaler

# Load the Iris dataset
iris = load_iris()
X = iris.data
y = iris.target
```

```
# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Supervised Learning: Linear Regression
# Train the model
linear_reg_model = LinearRegression()
linear_reg_model.fit(X_train, y_train)

# Supervised Learning: Logistic Regression
# Train the model
logreg_model = LogisticRegression(max_iter=1000)
logreg_model.fit(X_train, y_train)

# K-fold Cross-Validation
kf = KFold(n_splits=5, shuffle=True, random_state=42)

# Cross-validation with decision tree
tree_model = DecisionTreeClassifier()
tree_cv_score = cross_val_score(tree_model, X, y, cv=kf)
print ("Cross-Validation Accuracy (Decision Tree):", tree_cv_score.mean())

# Unsupervised Learning: K-Means Clustering
# Standardize the data
scaler  = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Fit K-Means clustering
kmeans_model = KMeans(n_clusters=3, random_state=42)
kmeans_model.fit(X_scaled)

# Evaluation: Silhouette Score
silhouette_avg = silhouette_score(X_scaled, kmeans_model.labels_)
print ("Silhouette Score for K-Means Clustering:", silhouette_avg)

# Plot results
plt.figure(figsize=(12, 6))

# Plot for Linear Regression
plt.subplot(1, 2, 1)
plt.plot(y_test, linear_reg_model.predict(X_test), 'bo-', label='Linear Regression')
plt.plot(y_test, y_test, 'r--', label='Ideal Prediction')
plt.title("Linear Regression")
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.legend()

# Plot for Logistic Regression
plt.subplot(1, 2, 2)
plt.plot(y_test, logreg_model.predict(X_test), 'go-', label='Logistic Regression')
```

```
plt.plot(y_test, y_test, 'r--', label='Ideal Prediction')
plt.title("Logistic  Regression")
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.legend()

plt.tight_layout()
plt.show()
```

**OUTPUT:**




**RESULT:**

The results for the above program User exploring machine learning techniques with real-world data was written, verified and executed successfully.

**EX.NO:7** **INSTALLATION OF HADOOP ENVIRONMENT**
**DATE:**

**AIM**

The aim of the program is installation of Hadoop framework and study of Hadoop ecosystem

**ALGORITHM / PROCEDURE**

**Installing Hadoop Environment**

The installation of a Hadoop environment involves several steps and components. Here's a high-level overview suitable for a single-node setup, which is a good starting point for learning and development purposes. This guide assumes you're using a Unix-like environment (e.g., Linux, macOS).

**Prerequisites**

**Java:** Hadoop is written in Java, so you need to have Java installed on your machine.
**SSH**: For communication between Hadoop's nodes, even in a single-node setup.

**Steps**

1. **Install Java**

- ✓ Update your package index: **sudo apt-get** upDATE (Debian/Ubuntu) or **sudo yum update** (Fedora/CentOS).
- ✓ Install Java: s**udo apt-get install default-jdk (**Debian/Ubuntu) or **sudo yum install java-1.8.0-openjdk** (Fedora/CentOS).
- ✓ Verify the installation: **java -version**.

2. **Configure SSH**

- ✓ Install SSH (if it's not already installed): **sudo apt-get install openssh-server openssh-client** (Debian/Ubuntu) or **sudo yum install openssh-server openssh-clients (Fedora/CentOS).**
- ✓ Generate SSH keys, if you haven't already: **ssh-keygen -t rsa -P '' -f ~/.ssh/id_rsa.**
- ✓ Authorize SSH keys: **cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys and chmod 0600 ~/.ssh/authorized_keys.**

3. **Install Hadoop**

- ✓ Download the latest Hadoop release from the official website.
- ✓ Extract the archive: t**ar zxvf hadoop-x.y.z.tar.gz.**
- ✓ Move the Hadoop folder to a convenient location: **sudo mv hadoop-x.y.z /usr/local/hadoop.**

4. **Configure Hadoop**

- ✓ Set Java path in **hadoop-env.sh:** Edit **/usr/local/hadoop/etc/hadoop/hadoop-env.sh**, find the line **export JAVA_HOME=**, and set it to your Java installation path.
- ✓ Configure **core-site.xml, hdfs-site.xml, mapred-site.xml, and yarn-site.xml** according to your needs. Basic configurations can be found in Hadoop's documentation or tutorials online.

5. **Format the NameNode**: **hadoop namenode -format**.

6. **Start Hadoop**: Start all Hadoop daemons with **/usr/local/hadoop/sbin/start-all.sh.**

7. **Verify Installation:** Access the Hadoop web interface by visiting **http://localhost:50070/** from your browser.

**Study of Hadoop Ecosystem**

The Hadoop ecosystem includes a variety of tools that extend Hadoop's capabilities in processing, managing, and analyzing big data:

- **HDFS (Hadoop Distributed File System):** The primary storage system used by Hadoop applications.
- **MapReduce:** A programming model for processing large datasets with a parallel, distributed algorithm on a cluster.
- **YARN (Yet Another Resource Negotiator):** Manages resources of the systems storing the data and running the analysis.
- **Hive:** A data warehouse infrastructure that provides data summarization and ad hoc querying.
- **Pig:** A high-level platform for creating MapReduce programs used with Hadoop.
- **HBase:** A non-relational (NoSQL) database that runs on top of HDFS.
- **Sqoop:** A tool designed for efficiently transferring bulk data between Hadoop and structured datastores such as relational databases.
- **Flume:** A service for efficiently collecting, aggregating, and moving large amounts of log data to HDFS.
- **Oozie:** A workflow scheduler system to manage Hadoop jobs.

**Zookeeper:** A centralized service for maintaining configuration information, naming, providing distributed synchronization, and providing group services.

**EX.NO:8** **IMPLEMENT WORD COUNT USING MAPREDUCE**

**DATE:**

**AIM**

To implement a simple MapReduce algorithm for word count in Python using concurrent execution with ThreadPoolExecutor.

**ALGORITHM / PROCEDURE**

1) The mapper function takes a text string as input, splits it into words, and creates a dictionary where the keys are words and the values are their count.
2) The reducer function takes an item, which is a tuple containing a word and a list of its counts, and returns a tuple with the word and the sum of its counts.
3) The map_reduce function orchestrates the MapReduce process. It first maps each input data using the mapper function in parallel, then reduces the mapped data using the reducer function.
4) The if__name__== "_main_": block serves as the entry point of the script. It defines the sample input data and executes the MapReduce process on that data, printing the result.

**PROGRAM:**

```
from collections import defaultdict
from concurrent.futures import ThreadPoolExecutor
def mapper(text):
    words = text.split()
    word_count = defaultdict(int)
    for word in words:
        word_count[word] += 1
    return word_count.items()
def reducer(item):
    word, counts = item
    return word, sum(counts)
def map_reduce(data):
    # Map phase
    with ThreadPoolExecutor() as executor:
        mapped_data = list(executor.map(mapper, data))
     # Reduce phase
    reduced_data = defaultdict(list)
    for sublist in mapped_data:
        for item in sublist:
            word, count = item
            reduced_data[word].append(count)
       with ThreadPoolExecutor() as executor:
        reduced_data = list(executor.map(reducer, reduced_data.items()))
        return reduced_data
if__name__== "_main__":
    # Sample input data
    data = [    "hello world",    "world hello",    "hello hello world"    ]
    result = map_reduce(data)
    print(result)
```

**OUTPUT:**

```
[('hello', 4), ('world', 3)]
```

**RESULT:**

Thus, the above program implemented word count using map reduce was successfully completed.

**EX.NO:9** **HADOOP MAPREDUCE/PYSPARK**

**DATE:**

**AIM**

To Implementing algorithms like matrix multiplication in MapReduce.

**ALGORITHM / PROCEDURE**

**1. Mapper Input:**

The input to the mapper will be the row index, column index, and value from both matrices A and B.

Each input record could be in the format (matrixID, rowIndex, columnIndex, value), where matrixID is 'A' or 'B' indicating the matrix to which the value belongs.

**2. Map Phase:**

For each element from matrix A, emit a key-value pair for each column index in the result matrix C. The key is a tuple (rowIndex of A, columnIndex of C), and the value is a tuple (matrixID, columnIndex of A, value).

For each element from matrix B, emit a key-value pair for each row index in the result matrix C. The key is a tuple (rowIndex of C, columnIndex of B), and the value is a tuple (matrixID, rowIndex of B, value).

**3. Reduce Phase:**

The reducer receives all values for each key, which includes all necessary elements to compute a cell in matrix C.

For each key (i, j), compute the sum of the product of corresponding elements from A and B that share the same inner dimension index. This involves multiplying values from A and B where the columnIndex of A matches the rowIndex of B, and summing these products to get the final value for C[i][j].

**4. Output:**

The reducer outputs the final computed value for each cell in matrix C, along with its row and column indices.

**PROGRAM:**

```
from pyspark import SparkContext
sc = SparkContext()
# Example matrices A and B
# Represented as ((row, col), value)
matrix_a = [((0, 0), 1), ((0, 1), 2),
        ((1, 0), 3), ((1, 1), 4)]
matrix_b = [((0, 0), 5), ((0, 1), 6),
        ((1, 0), 7), ((1, 1), 8)]
# Parallelize matrices
rdd_a = sc.parallelize(matrix_a)
rdd_b = sc.parallelize(matrix_b)
# Map step
mapped_a = rdd_a.flatMap(lambda x: [((x[0][0], i), ('A', x[0][1], x[1])) for i in range(2)])
mapped_b = rdd_b.flatMap(lambda x: [((i, x[0][1]), ('B', x[0][0], x[1])) for i in range(2)])
```

```
# Reduce step
result_rdd = (mapped_a.union(mapped_b)
        .groupByKey()
        .map(lambda x: (x[0], sum([val[1]*val[2] for val in list(x[1]) if val[0]=='A']) *
sum([val[1]*val[2] for val in list(x[1]) if val[0]=='B']))))
result = result_rdd.collect()
print(result)
```

**OUTPUT:**

**[((1, 1), 32), ((0, 1), 16), ((0, 0), 14), ((1, 0), 28)]**

**RESULT:**

Thus, the above program Implementing algorithms like matrix multiplication in Map Reduce was successfully completed

**EX.NO:10** **INSTALL AND CONFIGURE NOSQL DATABASES**
**DATE:**

**AIM**
To Installing and configuring NoSQL databases like MongoDB, Cassandra, HBase, or Hypertable involves different steps based on the specific database system

**PROGRAM:**

**MongoDB**

**Installation:**

**On Ubuntu:**

*sudo apt-get upDATE*
*sudo apt-get install -y mongodb*

**Configuration:**

MongoDB runs with default settings suitable for development. For production, you might need to edit the configuration file, typically located at /etc/mongod.conf.

**Basic Commands:**

- **Start MongoDB:** sudo systemctl start mongodb
- **Verify MongoDB is running:** sudo systemctl status mongodb
- **Connect to MongoDB shell:** mongo
  - **Create or switch to a database:** use mydatabase
  - **Insert a document:** db.mycollection.insertOne({name: "John Doe", age: 30})
  - **Find documents:** db.mycollection.find()

**Cassandra Installation:**

**On Ubuntu:**

*echo "deb http://www.apache.org/dist/cassandra/debian 311x main" | sudo tee -a /etc/apt/sources.list.d/cassandra.sources.list*
*curl https://www.apache.org/dist/cassandra/KEYS | sudo apt-key add -*
*sudo apt-get update*
*sudo apt-get install cassandra*

**Configuration:**

Configuration files are located in /etc/cassandra. The main configuration file is cassandra.yaml.

**Basic Commands:**

- **Start Cassandra: s**udo service cassandra start
- **Verify it's running:** nodetool status
- **Connect to Cassandra shell:** cqlsh
  - **Create a keyspace:** CREATE KEYSPACE mykeyspace WITH replication = {'class':'SimpleStrategy', 'replication_factor' : 3};
  - **Use keyspace:** USE mykeyspace;

- **Create a table:** CREATE TABLE mytable (id int PRIMARY KEY, name text, age int);
- **Insert data:** INSERT INTO mytable (id, name, age) VALUES (1, 'John Doe', 30);
- **Query data:** SELECT * FROM mytable;

**HBase Installation:**

HBase requires Hadoop as a prerequisite. Once Hadoop is set up:

- Download HBase from HBase's website and extract it.
- Configure HBase in standalone mode by editing conf/hbase-site.xml.

**Configuration:**

For a basic setup, edit hbase-site.xml to define the file system and directory where HBase will store data.

**Basic Commands:**

- **Start HBase:** *start-hbase.sh*
- **Access HBase shell:** *hbase shell*
- **List tables:** *list*
- **Create a table:** *create 'mytable', 'mycf'*
- **Put data:** *put 'mytable', 'row1', 'mycf:name', 'John Doe'*
- **Get data:** *get 'mytable', 'row1'*
- **Scan table:** *scan 'mytable'*

**MongoDB**

MongoDB is a document-oriented database that stores data in JSON-like formats. Here are some basic operations:

1. Connect to MongoDB Shell: Run mongo in your terminal.
2. Create or Switch Database:
   *use myDatabase*
3. Insert Documents:
   *db.myCollection.insertOne({name: "John Doe", age: 30})*
4. Find Documents:
   *db.myCollection.find({name: "John Doe"})*
5. Update Documents:
   *db.myCollection.updateOne({name: "John Doe"}, {$set: {age: 31}})*
6. Delete Documents:
   *db.myCollection.deleteOne({name: "John Doe"})*

## Cassandra

Cassandra is a wide-column store designed to handle large amounts of data across many commodity servers.

1. Connect to Cassandra: Run cqlsh in your terminal.
2. Create Keyspace (similar to a database in relational databases):
   *CREATE KEYSPACE myKeyspace WITH replication = {'class': 'SimpleStrategy', 'replication_factor' : 3};*

3. Use Keyspace:
   USE myKeyspace;
4. Create Table:
   CREATE TABLE myTable (id UUID PRIMARY KEY, name TEXT, age INT);
5. Insert Data:
   INSERT INTO myTable (id, name, age) VALUES (uuid(), 'John Doe', 30);
6. Query Data:
   SELECT * FROM myTable WHERE name = 'John Doe';
7. Up Date Data:
   UPDATE myTable SET age = 31 WHERE id = <some-uuid>;
8. Delete Data:

   *DELETE FROM myTable WHERE id = <some-uuid>;*

**HBase**

HBase is a distributed column-oriented database built on top of the Hadoop file system.

1. Start HBase Shell: Run hbase shell in your terminal.
2. List Tables:
   *list*
3. Create Table:
   *create 'myTable', 'myFamily'*
4. Insert Data:
   *put 'myTable', 'row1', 'myFamily:name', 'John Doe'*
5. Retrieve Data:
   *get 'myTable', 'row1'*
6. Scan Table:
   *scan 'myTable'*
7. Delete Data:
   *delete 'myTable', 'row1', 'myFamily:name'*
8. Drop Table:
   *disable 'myTable'*
   *drop 'myTable'*

**RESULT:**

Thus, the above program Install and Configure NoSQL Databaseswas successfully completed

## EX.NO:11 QUERIES TO SORT AND AGGREGATE THE DATA IN A TABLE USING HIVEQL

**DATE:**

**AIM**

To Write queries to sort and aggregate the data in a table using HiveQL

**DESCRIPTION:**

Hive is an open-source data warehousing solution built on top of Hadoop. It supports an SQL-like query language called HiveQL. These queries are compiled into MapReduce jobs that are executed on Hadoop. While Hive uses Hadoop for execution of queries, it reduces the effort that goes into writing and maintaining MapReduce jobs. Hive supports database concepts like tables, columns, rows and partitions. Both primitive (integer, float, string) and complex data-types(map, list, struct) are supported. Moreover, these types can be composed to support structures of arbitrary complexity. The tables are serialized/deserialized using default serializers/deserializer. Any new data format and type can be supported by implementing SerDe and ObjectInspector java interface. HiveQL - ORDER BY and SORT BY Clause By using HiveQL ORDER BY and SORT BY clause, we can apply sort on the column. It returns the result set either in ascending or descending order. Here, we are going to execute these clauses on the records of the below table:

**emp**

| Id | Name | Salary | Department |
|---|---|---|---|
| 1 | Gaurav | 30000 | Developer |
| 2 | Aryan | 20000 | Manager |
| 3 | Vishal | 40000 | Manager |
| 4 | John | 10000 | Trainer |
| 5 | Henry | 25000 | Developer |
| 6 | William | 9000 | Developer |
| 7 | Lisa | 25000 | Manager |
| 8 | Ronit | 20000 | Trainer |

**HiveQL - ORDER BY Clause**

In HiveQL, ORDER BY clause performs a complete ordering of the query result set. Hence, the complete data is passed through a single reducer. This may take much time in the execution of large datasets. However, we can use LIMIT to minimize the sorting time.

Example: Select the database in which we want to create a table.

*hive> use hiveql;*

```
codegyani@ubuntu64server: ~
hive> use hiveql;
OK
Time taken: 0.067 seconds
hive>
```

Now, create a table by using the following command:
*hive> create table emp (Id int, Name string , Salary float, Department string) row format delimited*
*fields terminated by ',' ;*

```
codegyani@ubuntu64server: ~
hive> create table emp (Id int, Name string , Salary float, Department string)

    > row format delimited
    > fields terminated by ',' ;
OK
Time taken: 0.419 seconds
hive>
```

Load the data into the table
*hive> load data local inpath '/home/codegyani/hive/emp_data' into table emp;*

```
codegyani@ubuntu64server: ~
hive> load data local inpath '/home/codegyani/hive/emp_data' into table emp;
Loading data to table hiveql.emp
Table hiveql.emp stats: [numFiles=1, totalSize=200]
OK
Time taken: 1.411 seconds
hive>
```

Now, fetch the data in the descending order by using the following command
*hive> select * from emp order by salary desc;*

```
codegyani@ubuntu64server: ~
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2019-08-02 06:36:48,908 Stage-1 map = 0%,  reduce = 0%
2019-08-02 06:37:49,829 Stage-1 map = 0%,  reduce = 0%
2019-08-02 06:38:02,548 Stage-1 map = 100%,  reduce = 0%, Cumulative CPU 7.03 se
c
2019-08-02 06:39:03,090 Stage-1 map = 100%,  reduce = 0%, Cumulative CPU 10.31 s
ec
2019-08-02 06:39:18,347 Stage-1 map = 100%,  reduce = 67%, Cumulative CPU 12.98
sec
2019-08-02 06:39:35,537 Stage-1 map = 100%,  reduce = 100%, Cumulative CPU 22.34
 sec
MapReduce Total cumulative CPU time: 22 seconds 340 msec
Ended Job = job_1555046592674_0032
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1  Reduce: 1   Cumulative CPU: 22.34 sec   HDFS Read: 6788 H
DFS Write: 227 SUCCESS
```

```
codegyani@ubuntu64server: ~
hive> select * from emp order by salary desc;
Query ID = codegyani_20190802063522_65b28a82-4d0b-492a-ae25-2faef1471b65
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1555046592674_0032, Tracking URL = http://ubuntu64server:8088
/proxy/application_1555046592674_0032/
Kill Command = /home/codegyani/hadoop-2.7.1//bin/hadoop job  -kill job_155504659
2674_0032
```

```
codegyani@ubuntu64server: ~
Total MapReduce CPU Time Spent: 22 seconds 340 msec
OK
3       "Vishal"        40000.0 Manager
1       "Gaurav"        30000.0 Developer
7       "Lisa"  25000.0 Manager
5       "Henry" 25000.0 Developer
8       "Ronit" 20000.0 Trainer
2       "Aryan" 20000.0 Manager
4       "John"  10000.0 Trainer
6       "William"       9000.0  Developer
NULL    NULL    NULL    NULL
Time taken: 257.304 seconds, Fetched: 9 row(s)
hive>
```

**HiveQL - SORT BY Clause**

The HiveQL SORT BY clause is an alternative of ORDER BY clause. It orders the data within each reducer. Hence, it performs the local ordering, where each reducer's output is sorted separately. It may also give a partially ordered result. Example:

Let's fetch the data in the descending order by using the following command

*hive> select * from emp sort by salary desc;*

```
codegyani@ubuntu64server: ~
hive> select * from emp sort by salary desc;
Query ID = codegyani_20190802065014_f877314f-8d92-428f-8a9f-1b6a9b67c328
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks not specified. Estimated from input data size: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1555046592674_0033, Tracking URL = http://ubuntu64server:8088
/proxy/application_1555046592674_0033/
Kill Command = /home/codegyani/hadoop-2.7.1//bin/hadoop job  -kill job_155504659
2674_0033
```

```
codegyani@ubuntu64server: ~
Total MapReduce CPU Time Spent: 22 seconds 690 msec
OK
3       "Vishal"        40000.0 Manager
1       "Gaurav"        30000.0 Developer
7       "Lisa"  25000.0 Manager
5       "Henry" 25000.0 Developer
8       "Ronit" 20000.0 Trainer
2       "Aryan" 20000.0 Manager
4       "John"  10000.0 Trainer
6       "William"       9000.0  Developer
NULL    NULL    NULL    NULL
Time taken: 268.62 seconds, Fetched: 9 row(s)
```

**Cluster By:**

Cluster By used as an alternative for both Distribute BY and Sort BY clauses in Hive-QL. Cluster BY clause used on tables present in Hive. Hive uses the columns in Cluster by to distribute the rows among reducers. Cluster BY columns will go to the multiple reducers.

- It ensures sorting orders of values present in multiple reducers

For example, Cluster By clause mentioned on the Id column name of the table employees_guru table. The output when executing this query will give results to multiple reducers at the back end. But as front end it is an alternative clause for both Sort By and Distribute By.

Example:

*SELECT Id, Name from employees_guru CLUSTER BY Id;*

```
hive> Select Id,Name from employees_guru CLUSTER BY Id;
Query ID = hadoop_20151105165000_72cedc06-a797-48b1-a120-
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks not
In order to change the ave
  set hive.exec.reducers.b
In order to limit the maxi
  set hive.exec.reducers.m
In order to set a constant
  set mapred.reduce.tasks=<number>
Starting Job = job_201511051442_0009, Tracking URL = http
Kill Command = /usr/local/hadoop-1.2.1/libexec/../bin/had
Hadoop job information for Stage-1: number of mappers: 1;
2015-11-05 16:50:08,541 Stage-1 map = 0%,  reduce = 0%
2015-11-05 16:50:10,546                    reduce = 0%,
2015-11-05 16:50:17,563                    reduce = 100
MapReduce Total cumulat                    nds 600 msec
Ended Job = job_20151
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1  Reduce: 1   Cumulative CPU: 1.6 se
Total MapRe  e CPU Time Spent: 1 seconds 600 msec
OK
101     Rajesh
102     Rajiv
103     Animesh
104     Anirudh
105     Santosh
106     Ramesh
107     Sravanthi
108     Sravan
109     Suresh
110     Ravi
111     Syam
Time taken: 18.941 seconds, Fetched: 11 row(s)
```

cluster by query

cluster by query output

**RESULT:**

Thus, the above program sort and aggregate the data in a table using HiveQL was successfully completed

**EX.NO:12 CRAFT A CODEBASE FOR SOCIAL MEDIA DATA VISUALIZATION**
**DATE:**

**AIM**

The aim of the algorithm is to retrieve tweets from a specific Twitter user's timeline using the Tweepy library in Python. By providing the username of the Twitter account as input, the algorithm authenticates with the Twitter API, retrieves the specified number of tweets from the user's timeline, and then prints out each tweet. This allows users to access and analyze tweets from a particular user programmatically, which can be useful for various purposes such as sentiment analysis, trend analysis, or simply for monitoring specific accounts.

**ALGORITHM / PROCEDURE**

1. Import the tweepy library

2. Define function get_tweets(username):
   a. Initialize consumer_key, consumer_secret, access_key, and access_secret with your Twitter API credentials
   b. Authenticate with Twitter API using tweepy.OAuthHandler and set access token with set_access_token method
   c. Create API object using tweepy.API(auth)
   d. Set number_of_tweets to the desired number of tweets to be extracted
   e. Use api.user_timeline method to fetch tweets for the given username, providing screen_name=username and count=number_of_tweets as parameters
   f. Initialize an empty list tweet_list to store tweets
   g. Loop through each tweet in tweets fetched:
      i. Append the text of each tweet to tweet_list
   h. Print each tweet in tweet_list

3. Define main function:
   a. Call get_tweets function with the desired Twitter handle as argument

4. Execute main function if this script is run as the main program

**PROGRAM:**

```
from mpl toolkits. mplot3d import Axes3D
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt # plotting
import numpy as np # linear algebra
import os # accessing directory structure
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Distribution graphs (histogram/bar graph) of column data
def plotPerColumnDistribution(df, nGraphShown, nGraphPerRow):
    nunique = df.nunique()
    df = df[[col for col in df if nunique[col] > 1 and nunique[col] < 50]] # For displaying
```

```
purposes, pick columns that have between 1 and 50 unique values
    nRow, nCol = df.shape
    columnNames = list(df)
    nGraphRow = (nCol + nGraphPerRow - 1) / nGraphPerRow
    plt.figure(num = None, figsize = (6 * nGraphPerRow, 8 * nGraphRow), dpi = 80, facecolor
= 'w', edgecolor = 'k')
    for i in range(min(nCol, nGraphShown)):
        plt.subplot(nGraphRow, nGraphPerRow, i + 1)
        columnDf = df.iloc[:, i]
        if (not np.issubdtype(type(columnDf.iloc[0]), np.number)):
            valueCounts = columnDf.value_counts()
            valueCounts.plot.bar()
        else:
            columnDf.hist()
        plt.ylabel('counts')
        plt.xticks(rotation = 90)
        plt.title(f'{columnNames[i]} (column {i})')
    plt.tight_layout(pad = 1.0, w_pad = 1.0, h_pad = 1.0)
    plt.show()

    # Correlation matrix
    def plotCorrelationMatrix(df, graphWidth):
        filename = df.dataframeName
        df = df.dropna('columns') # drop columns with NaN
        df = df[[col for col in df if df[col].nunique() > 1]] # keep columns where there are more
than 1 unique values
        if df.shape[1] < 2:
            print(f'No correlation plots shown: The number of non-NaN or constant columns
({df.shape[1]}) is less than 2')
            return
        corr = df.corr()
        plt.figure(num=None, figsize=(graphWidth, graphWidth), dpi=80, facecolor='w',
edgecolor='k')
        corrMat = plt.matshow(corr, fignum = 1)
        plt.xticks(range(len(corr.columns)), corr.columns, rotation=90)
        plt.yticks(range(len(corr.columns)), corr.columns)
        plt.gca().xaxis.tick_bottom()
        plt.colorbar(corrMat)
        plt.title(f'Correlation Matrix for {filename}', fontsize=15)
        plt.show()

    # Scatter and density plots
    def plotScatterMatrix(df, plotSize, textSize):
```

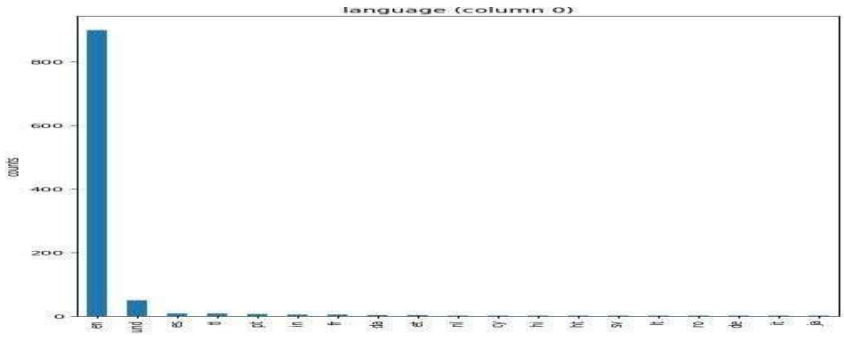
```
    df = df.select_dtypes(include =[np.number]) # keep only numerical columns
    # Remove rows and columns that would lead to df being singular
    df = df.dropna('columns')
    df = df[[col for col in df if df[col].nunique() > 1]] # keep columns where there are more than 1 unique values
    columnNames = list(df)
    if len(columnNames) > 10: # reduce the number of columns for matrix inversion of kernel density plots
        columnNames = columnNames[:10]
    df = df[columnNames]
    ax = pd.plotting.scatter_matrix(df, alpha=0.75, figsize=[plotSize, plotSize], diagonal='kde')
    corrs = df.corr().values
    for i, j in zip(*plt.np.triu_indices_from(ax, k = 1)):
        ax[i, j].annotate('Corr. coef = %.3f' % corrs[i, j], (0.8, 0.2), xycoords='axes fraction', ha='center', va='center', size=textSize)
    plt.suptitle('Scatter and Density Plot')
    plt.show()

nRowsRead = 1000 # specify 'None' if want to read whole file
# tweets.csv may have more rows in reality, but we are only loading/previewing the first 1000 rows
df1 = pd.read_csv('extracted_tweets.csv', delimiter=',', nrows = nRowsRead)
df1.dataframeName = 'tweets.csv'
nRow, nCol = df1.shape
print(f'There are {nRow} rows and {nCol} columns')
```

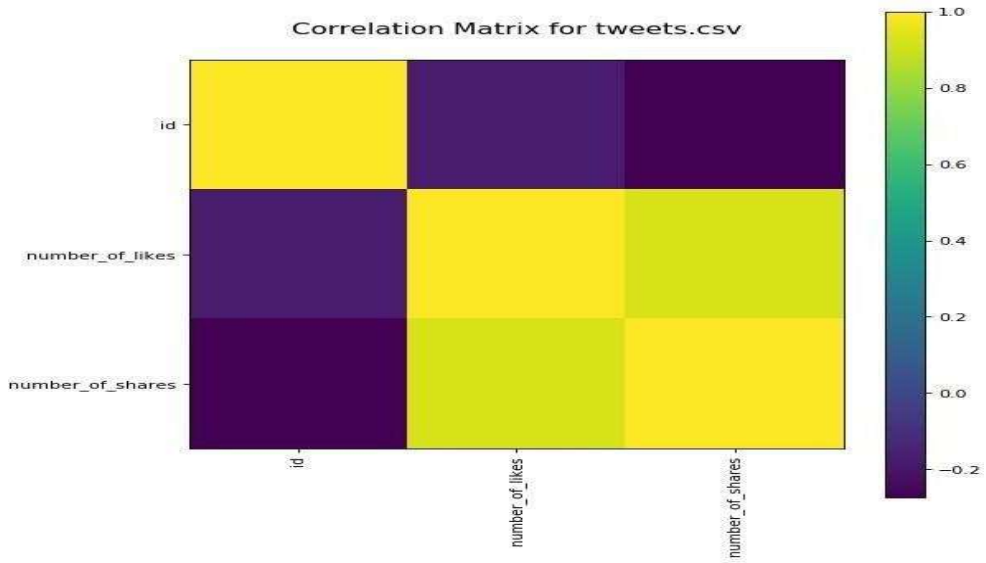
**Tweepy :**

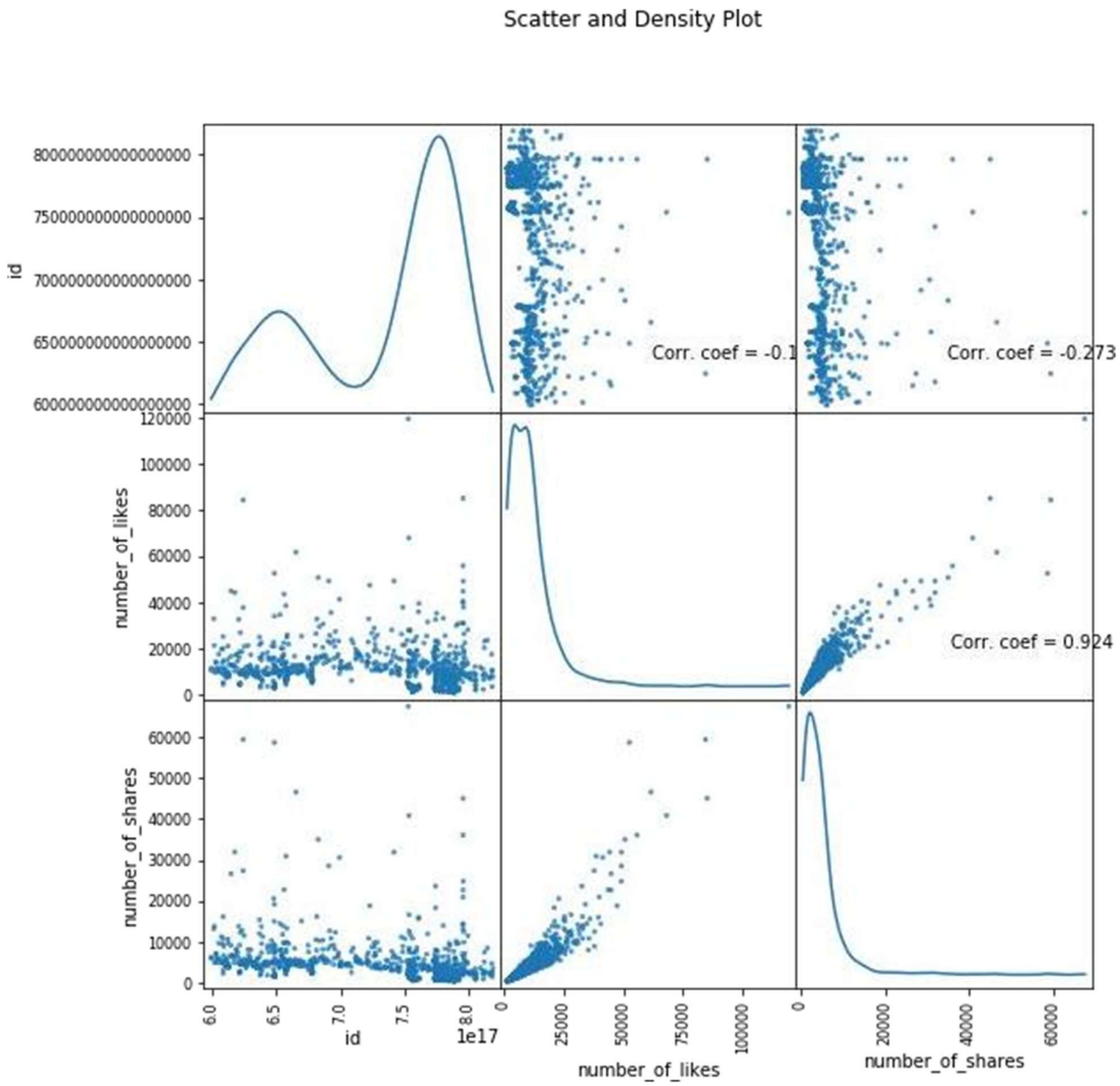
```
import tweepy

# Fill the X's with the credentials obtained by
# following the above mentioned procedure.
consumer_key = "your-generated-key"
consumer_secret = "your-generated-secret"
access_key = "your-developer-key"
access_secret = "your-developer-secret"

# Function to extract tweets
def get_tweets(username):

        # Authorization to consumer key and consumer secret
        auth = tweepy.OAuthHandler(consumer_key, consumer_secret)
```

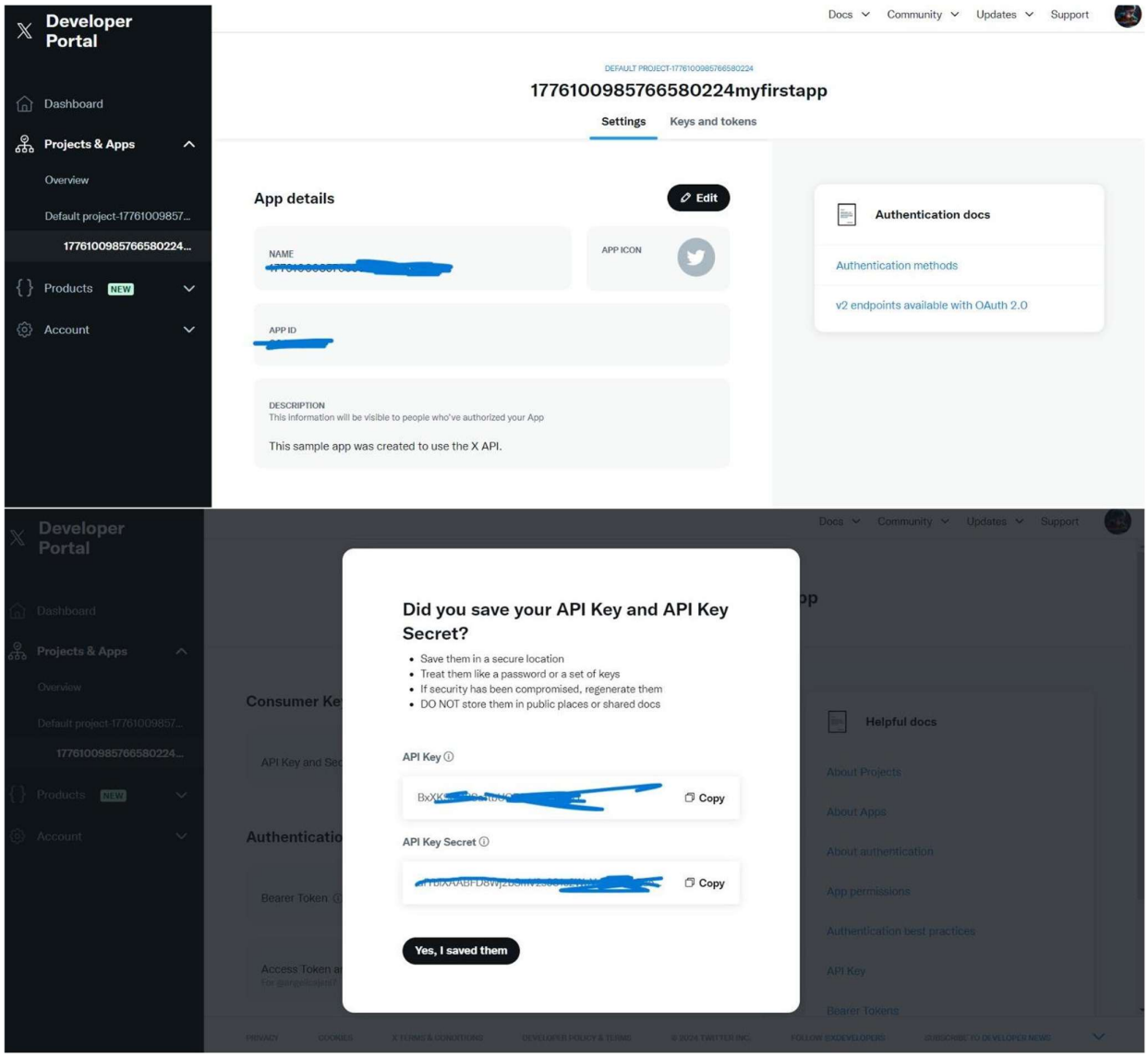
```
        # Access to user's access key and access secret
        auth.set_access_token(access_key, access_secret)

        # Calling api
        api = tweepy.API(auth)

        # 200 tweets to be extracted
        number_of_tweets=200
        tweets = api.user_timeline(screen_name=username)

        # Empty Array
         tmp=[]

        # create array of tweet information: username,
        # tweet id, date/time, text
        tweets_for_csv = [tweet.text for tweet in tweets] # CSV file created
        for j in tweets_for_csv:

            # Appending tweets to the empty array tmp
            tmp.append(j)

        # Printing the tweets
        print(tmp)
# Driver code

if _name_ == '_main_':

    # Here goes the twitter handle for the user
    # whose tweets are to be extracted.
    get_tweets("your-twitter-handle-name")
```

**OUTPUT:**




Correlation Matrix for tweets.csv
id
number_of_likes
number_of_shares
1.0
0.8
0.6
0.4
0.2
0.0
−0.2


Scatter and Density Plot
id
number_of_likes
number_of_shares
Corr. coef = -0.1
Corr. coef = -0.273
Corr. coef = 0.924
1e17




**RESULT:**

Thus, the above program craft a codebase for social media data visualization was successfully completed.